

Tutorial_2.Rmd

Amelia Mesich

2025-08-11

Packages we will use

Packages are collections of useful functions, data and documentation written by other people as an extension of basic R features.

```
# a collection of R packages including ggplot, lubridate, forcats, stringr, tidyr, dplyr
#install.packages("tidyverse")
library(tidyverse)
```

```
## Warning: package 'tidyverse' was built under R version 4.3.3
```

```
## Warning: package 'tibble' was built under R version 4.3.2
```

```
## Warning: package 'readr' was built under R version 4.3.3
```

```
## Warning: package 'purrr' was built under R version 4.3.2
```

```
## Warning: package 'dplyr' was built under R version 4.3.2
```

```
## Warning: package 'stringr' was built under R version 4.3.2
```

```
## Warning: package 'forcats' was built under R version 4.3.3
```

```
## Warning: package 'lubridate' was built under R version 4.3.3
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
```

```
## v dplyr      1.1.4      v readr      2.1.5
```

```
## v forcats   1.0.0      v stringr   1.5.1
```

```
## v ggplot2    4.0.0      v tibble    3.2.1
```

```
## v lubridate  1.9.3      v tidyr     1.3.1
```

```
## v purrr      1.0.2
```

```
## -- Conflicts ----- tidyverse_conflicts() --
```

```
## x dplyr::filter() masks stats::filter()
```

```
## x dplyr::lag()     masks stats::lag()
```

```
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
#additional package that sets the working directory
```

```
#install.packages("here")
```

```
library(here)
```

```
## Warning: package 'here' was built under R version 4.3.2
```

```
## here() starts at C:/Users/ameli/OneDrive/Documents/CSB-345-tutorials
```

```
#additional package to change the scales on a time series plot  
#install.packages("scales")  
library(scales)
```

```
##  
## Attaching package: 'scales'  
##  
## The following object is masked from 'package:purrr':  
##  
##     discard  
##  
## The following object is masked from 'package:readr':  
##  
##     col_factor
```

```
#additional package to calculate statistical significance  
#install.packages("ggpubr")  
library(ggpubr)  
#we will use this to save out the results as a PDF  
#tinytex::install_tinytex()
```

```
##Section 1: Loading in the dataset
```

We will load in the data from the Bitsikas et al paper into a variable called fish_df. This is a subset of the total data and contains all the observations from the month of August.

```
fish_df <- read.csv(file = here("zebrafish_behaviour_subset.csv"))  
#examine how the data is structured  
str(fish_df)
```

```
## 'data.frame':    869486 obs. of  5 variables:  
## $ time      : chr  "2022-08-09 10:25:30" "2022-08-09 10:25:36" "2022-08-09 10:25:42" "2022-08-09 10:25:48" ...  
## $ Dist_6s   : num  0.433 0.387 0.477 0.326 0.239 ...  
## $ genotype  : chr   "WT" "WT" "WT" "WT" ...  
## $ daytime   : chr   "day" "day" "day" "day" ...  
## $ state     : chr   "Wake" "Wake" "Wake" "Wake" ...
```

We can see that this is a dataframe that contains the 869, 486 observations or rows.

Each observation has a timepoint (time column), the distance travelled in 6 seconds (Dist_6s column), the genotype of the individual (genotype column), if the observation occurred at day or night (daytime column) and the sleep-wake state of the individual (state column).

```
#look at the first 6 rows  
head(fish_df)
```

```
##           time      Dist_6s genotype daytime state  
## 1 2022-08-09 10:25:30 0.4327311      WT      day  Wake
```

```
## 2 2022-08-09 10:25:36 0.3873116 WT day Wake
## 3 2022-08-09 10:25:42 0.4768259 WT day Wake
## 4 2022-08-09 10:25:48 0.3259950 WT day Wake
## 5 2022-08-09 10:25:54 0.2390276 WT day Wake
## 6 2022-08-09 10:26:00 0.3167992 WT day Wake
```

Section 2: Convert format of date-time column

We will convert the time column to a date-time object using the function `as_datetime()` from `lubridate`.

We can also use the `day()`, `hour()` and `minute()` functions to extract the day, hour and minute from the time.

```
fish_df$time <- as_datetime(fish_df$time)
fish_df$day <- day(fish_df$time)
fish_df$hour <- hour(fish_df$time)
fish_df$minute <- minute(fish_df$time)
#fish_df$second <- second(fish_df$time)
#fish_df$timestamp <- paste(fish_df$hour, fish_df$minute, fish_df$second, sep = ":")
```

For the first plot, we will filter for just the wildtype (WT) data using the `filter()` function. We will call the subset dataframe `fish_WT`. This subsets the dataframe from 869,486 rows to 263,682 rows.

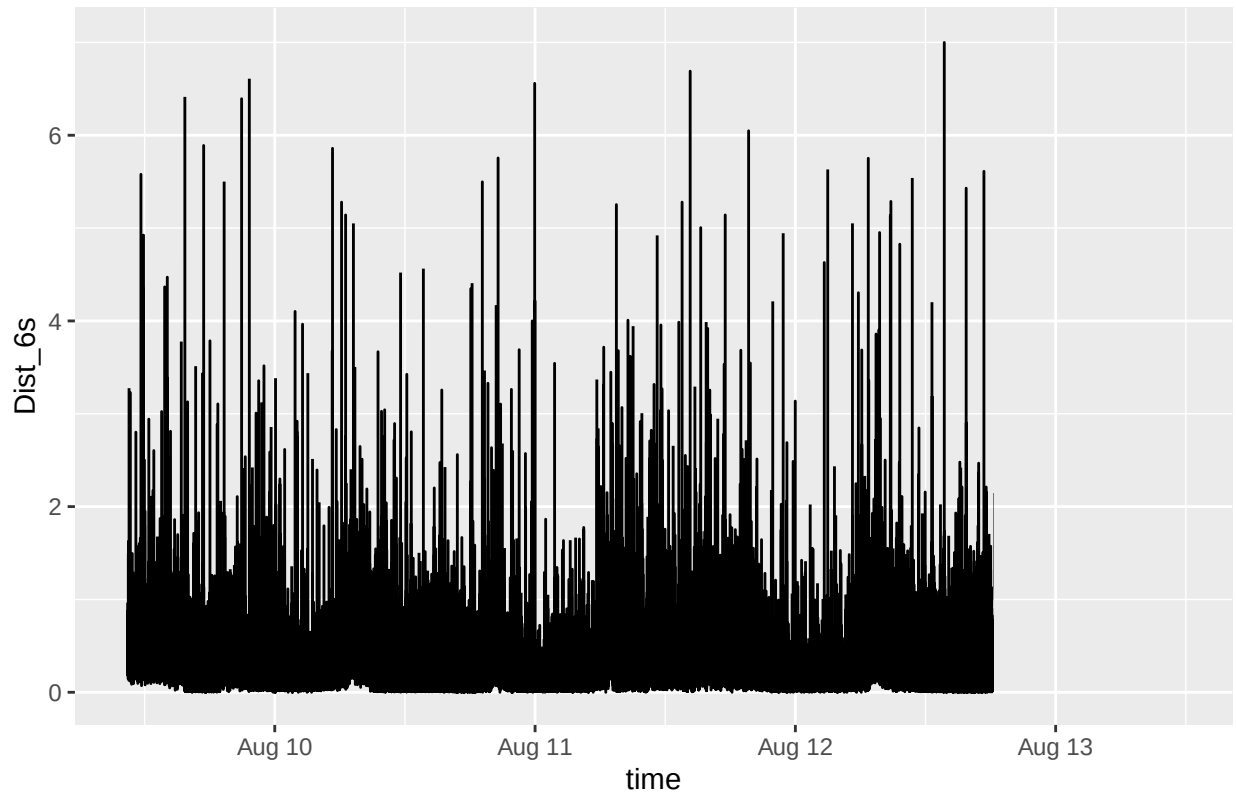
```
fish_WT <- filter(fish_df, genotype == "WT")
```

Section 3: Plotting the time series

The function `ggplot` allows us to create a customizable data visualizations. It requires the data to plot as the first variable, and the aesthetics to be used when making the plot (typically this is the x and y coordinates). You can then specify the type of plot you want to make with additional mapping layers. Here we use `geom_line()` but we will go through other examples of plots supported by `ggplot`.

```
#plot the raw observations (264k observations) from fish_WT
#set the x axis to be time and the y axis to be distance moved in 6s
ggplot(data = fish_WT, mapping = aes(x = time, y = Dist_6s)) + geom_line() + ggtitle("Plot 1: Zerbafish")
```

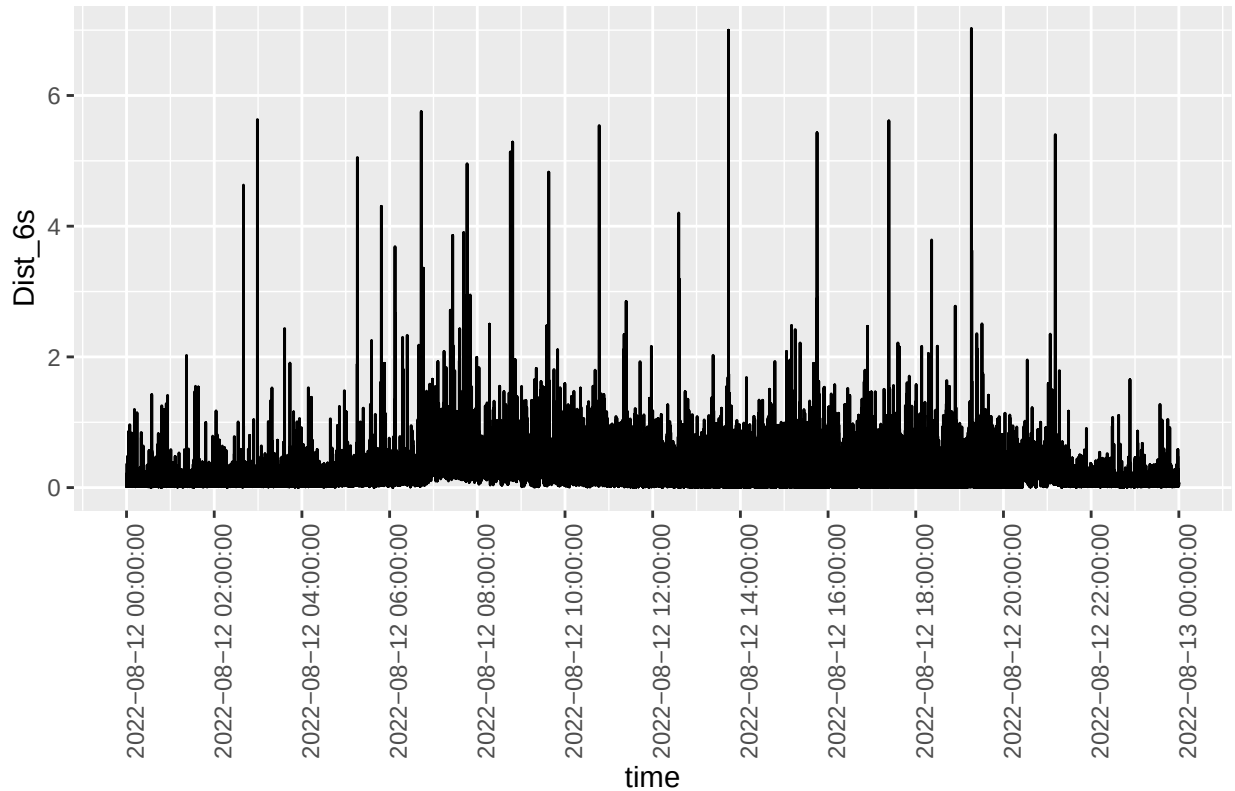
Plot 1: Zerbafish time series over 4 days



We can look at the data from one day of observations. By using the pipe operator `%>%` we can run this in one line. The pipe operator takes the output of one function and passes it to be the input of a second function and so on. First we pass the `fish_WT` dataframe to the `filter()` function and then pass the result of this (the filtered dataframe) to `ggplot` and create the same time series plot as above.

```
#plot all observations for one day  
fish_WT %>% filter(day == 12) %>% ggplot(., aes(x = time, y = Dist_6s)) + geom_line() + scale_x_datetime()
```


Plot 2: Zebrafish time series data over one day



We can compare this to the time series figure from the paper. We can see on both days there is more activity during the day and less at night. **Lights on occurs from 9:00:00 - 23:00:00 lights off occurs from 23:00:01 -8:59:59.

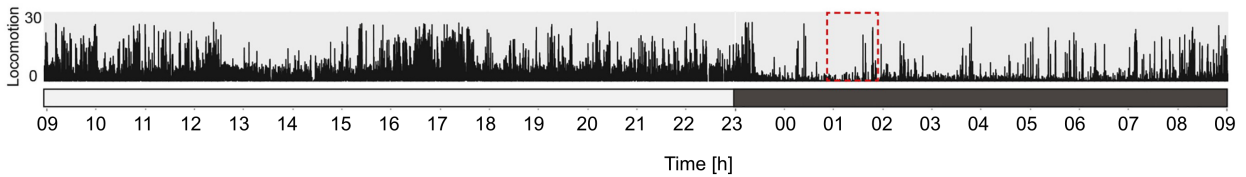


Figure 1: figure 1b.

Section 4: Determining sleep vs wake

Distance moved over time (speed) at any given time point is used to make calls on if a fish is asleep or awake. We can find what the cutoff they used is by looking at what the minimum speed they call wake is and what the maximum speed they call sleep is.

```
fish_wake <- fish_df %>% filter(state == "Wake")
min(fish_wake$Dist_6s)
```

```
## [1] 0.2000001
```

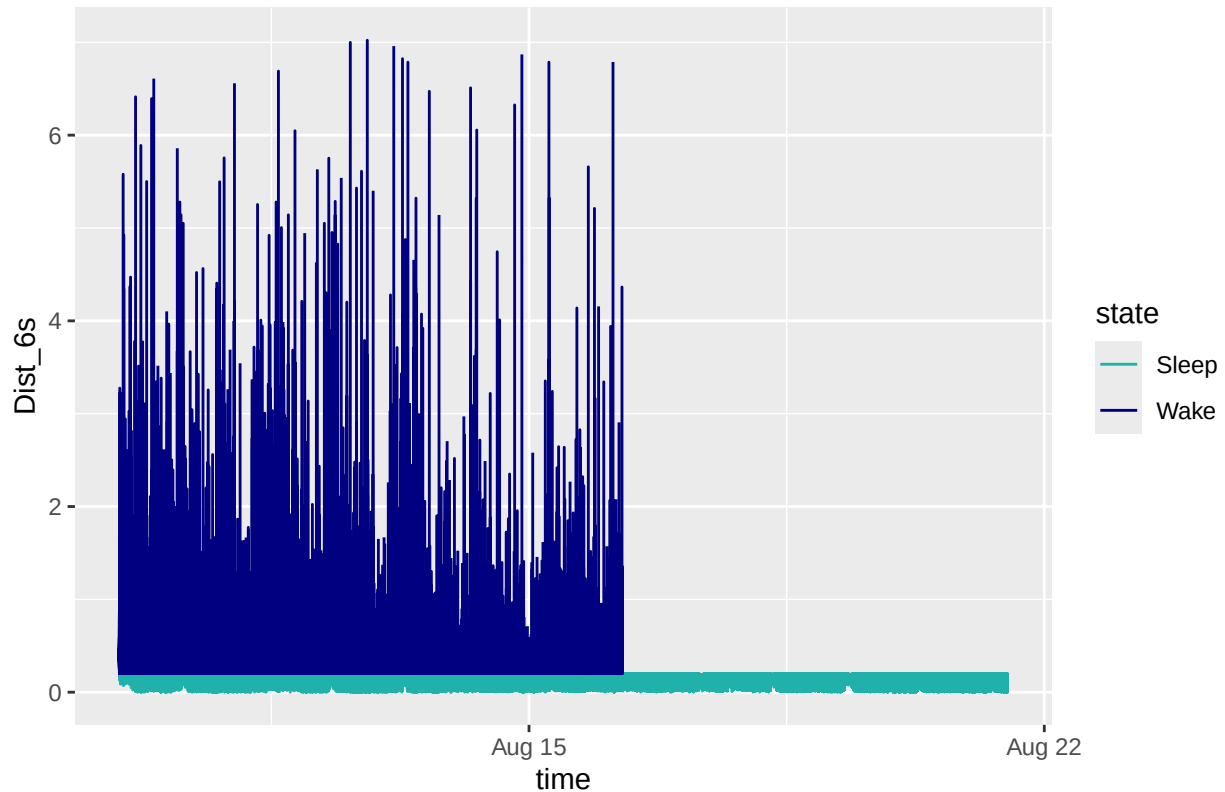
```
fish_sleep <- fish_df %>% filter(state == "Sleep")
max(fish_sleep$Dist_6s)
```

```
## [1] 0.1999996
```

It appears the cutoff they use is 0.2cm/s. We can see this cutoff when we plot the raw data as well.

```
ggplot(fish_df, aes(x = time, y = Dist_6s, colour = state)) + geom_line() + scale_colour_manual(values =
```

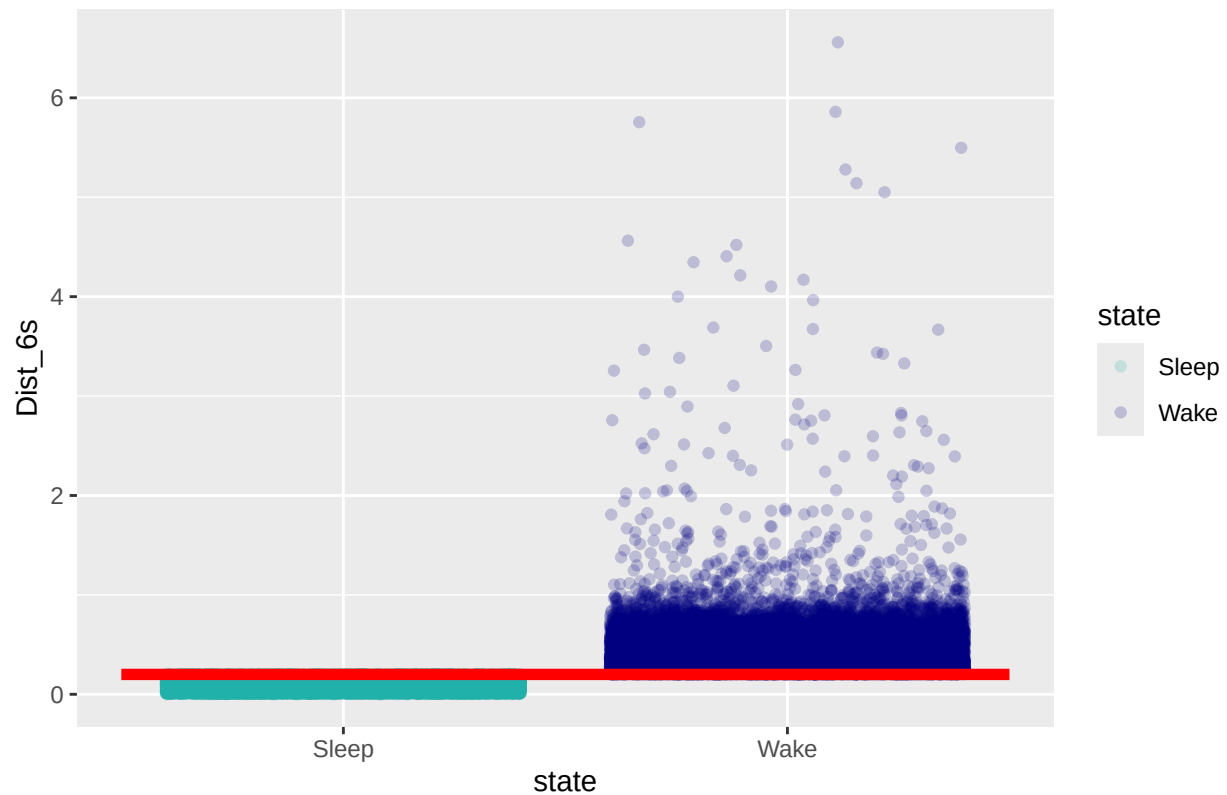
Plot 3: Zebrafish sleep vs wake speed over time



And when we summarize the data with boxplots.

```
fish_df %>% filter(day == 10) %>% ggplot(., aes(y = Dist_6s, x = state, colour = state)) + geom_jitter(
```

Plot 4: Zebrafish sleep vs wake cutoffs

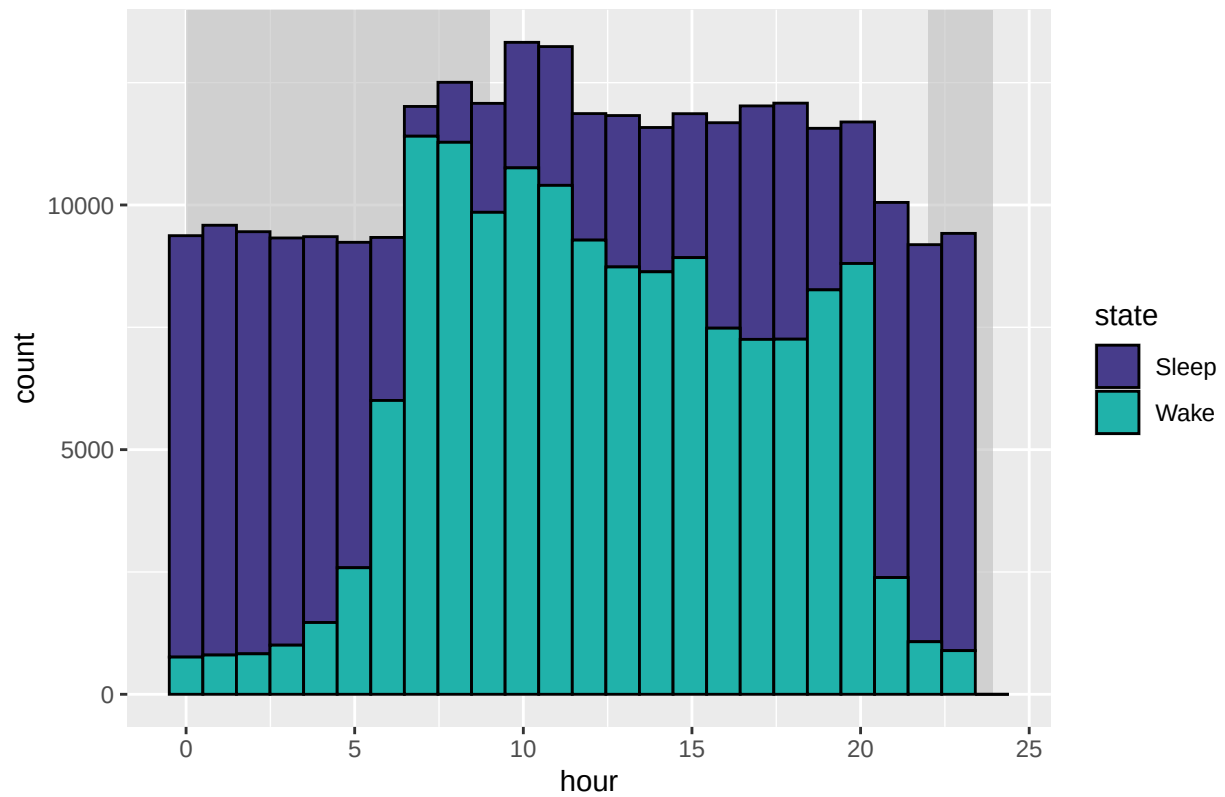


Section 5: Sleep timing in wildtype zebrafish

Zebrafish are diurnal; when we look at the total number of observations of sleep and wake, we can see that most sleep tends to occur during the night and most wake occurs during the day.

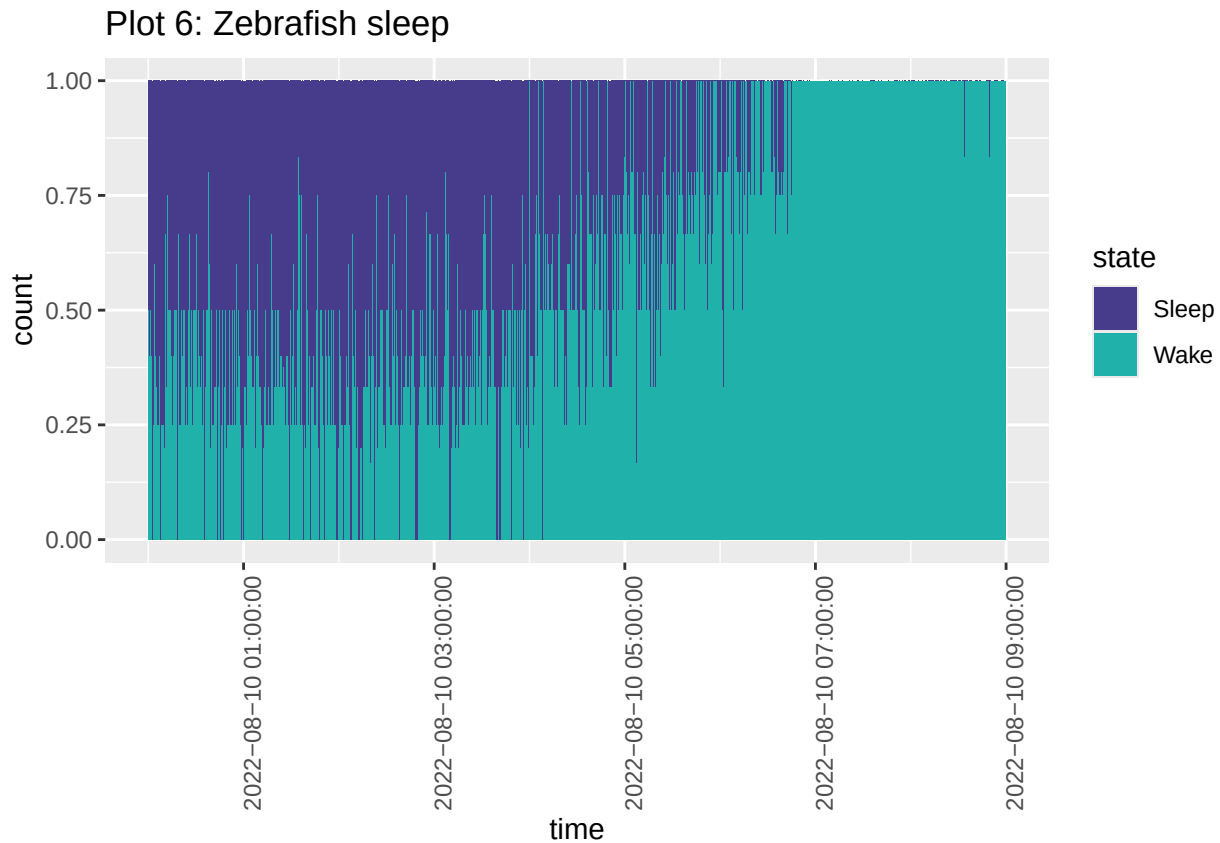
```
ggplot(fish_WT, aes(x = hour, fill = state)) + annotate("rect", xmin = 0, xmax = 9, ymin = 0, ymax = Inf, fill = "red", stroke = "black")
```

Plot 5: Total wake vs sleep observations in day vs night



In wildtype zebrafish sleep is consolidated.

```
fish_WT %>% filter(day == 10, hour %in% 0:8) %>% ggplot(., aes(x = time, fill = state)) + geom_bar(positi
```



Section 6: Comparison of sleep cycles in hypocretin knockouts vs wildtype

We want to compare the proportion of time sleep awake and the proportion of time spent asleep in the wildtype individuals vs the hypocretin knockouts.

To do this, we can use the `count()` function to count the number of observations for each state (wake vs sleep) in each hour (0-23) for each genotype (wildtype, homozygous KO, heterozygous KO). This create a new datafrom with a column called `n` which contains the counts for each group.

```
fish_counts <- fish_df %>% count(hour, state, genotype)
head(fish_counts)
```

```
##   hour state genotype    n
## 1    0 Sleep      Het 9804
## 2    0 Sleep      Hom 9107
## 3    0 Sleep      WT 8613
## 4    0 Wake      Het  979
## 5    0 Wake      Hom  789
## 6    0 Wake      WT  760
```

We now need to get the total number of sleep and wake observations in each genotype in each hour in order to later calculate the proportion of time spent in sleep and wake out of the total time observed. To do this we select the columns we need, then we group by only hour and genotype. This pools both the sleep and wake observations for each hour and each genotype. We can then sum together all the sleep and wake

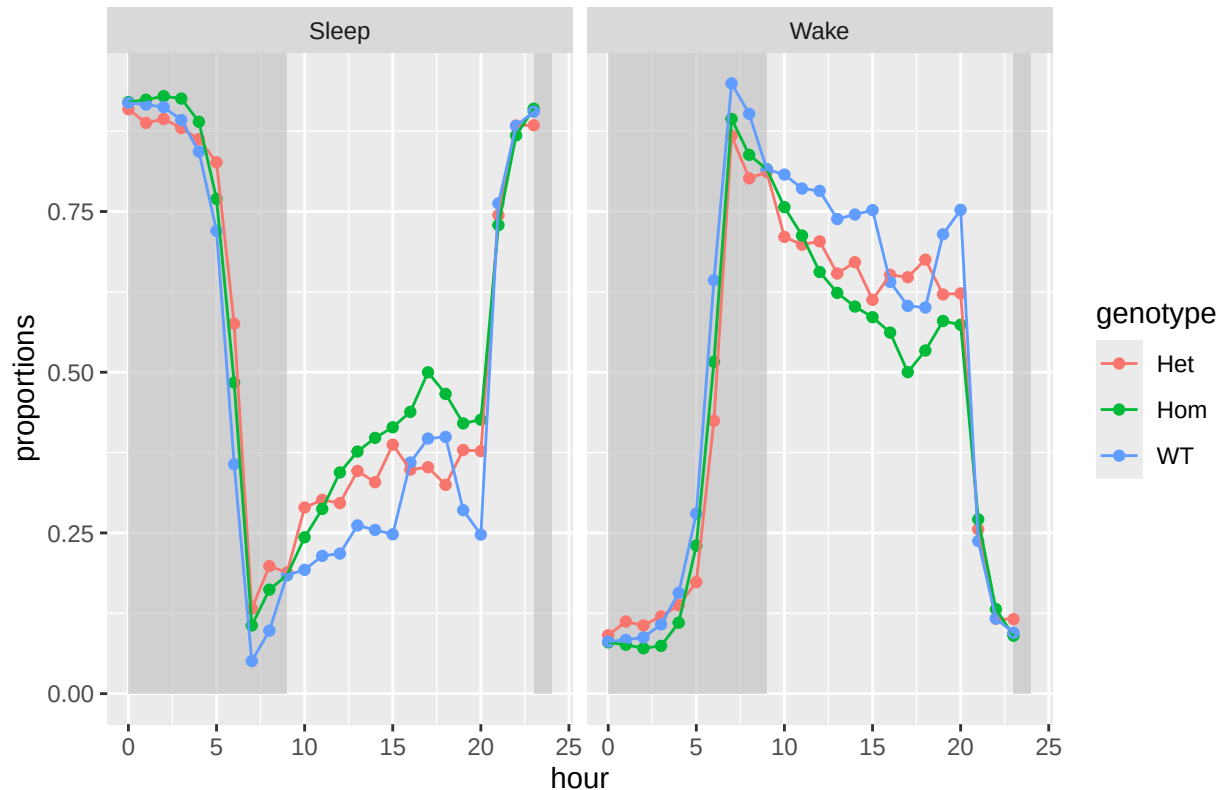
observations using mutate and the sum function. We then divide the number of sleep and wake observations by the total number of observations to get the proportions.

```
fish_counts <- fish_counts %>% select(hour, state, genotype, n) %>% group_by(hour, genotype) %>% mutate(
fish_counts$proportions <- fish_counts$n/fish_counts$totals
```

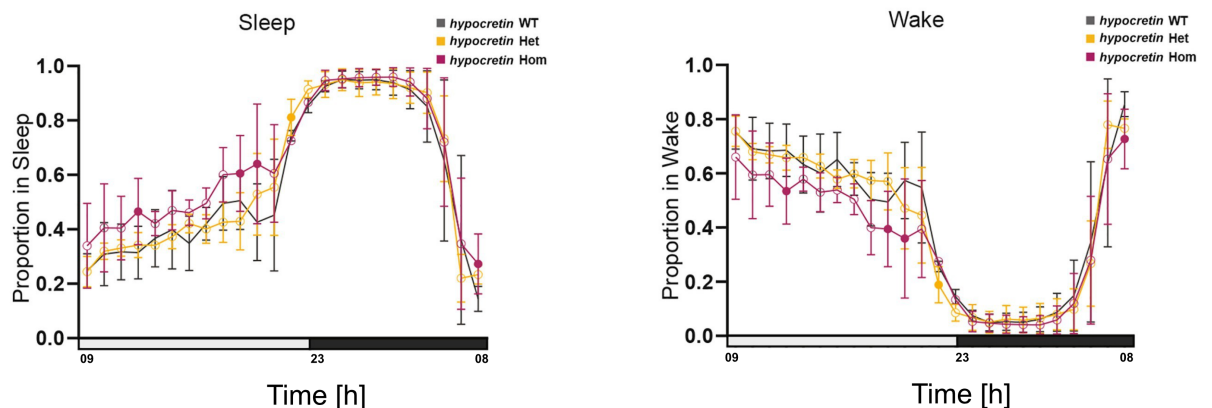
We can now plot the proportion of total time spent in sleep and wake for each genotype over 24 hours.

```
ggplot(fish_counts, aes(x = hour, y = proportions, color = genotype)) + annotate("rect", xmin = 0, xmax = 24, ymin = 0, ymax = 1, fill = "gray", alpha = 0.5)
```

Plot 7: Sleep and wake proportions in wildtype vs knockouts



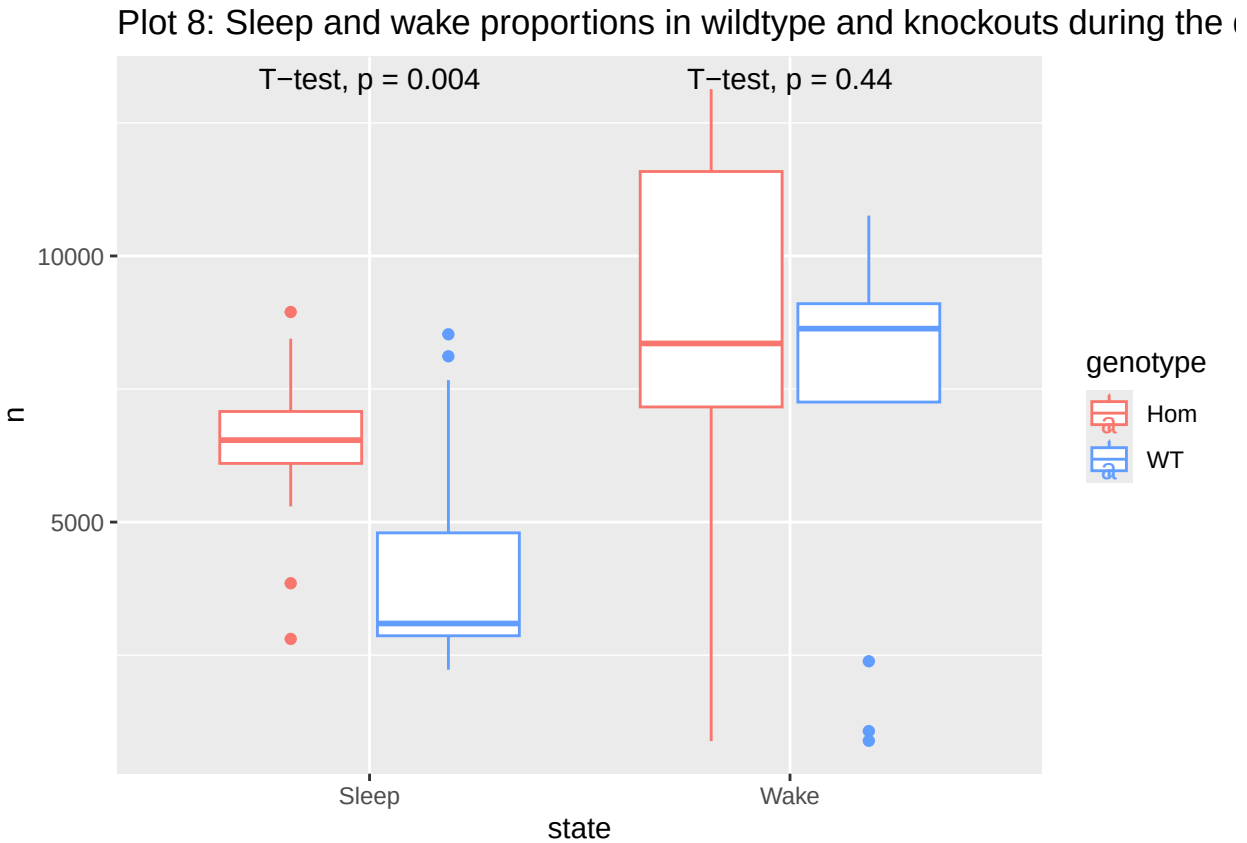
The homozygous hypocretin knockouts have more daytime sleep than wild type, the same result as we see in



the paper.

We can summarize this result with boxplots and use a t test to determine if there is a statistically significant difference between the mean of each group.

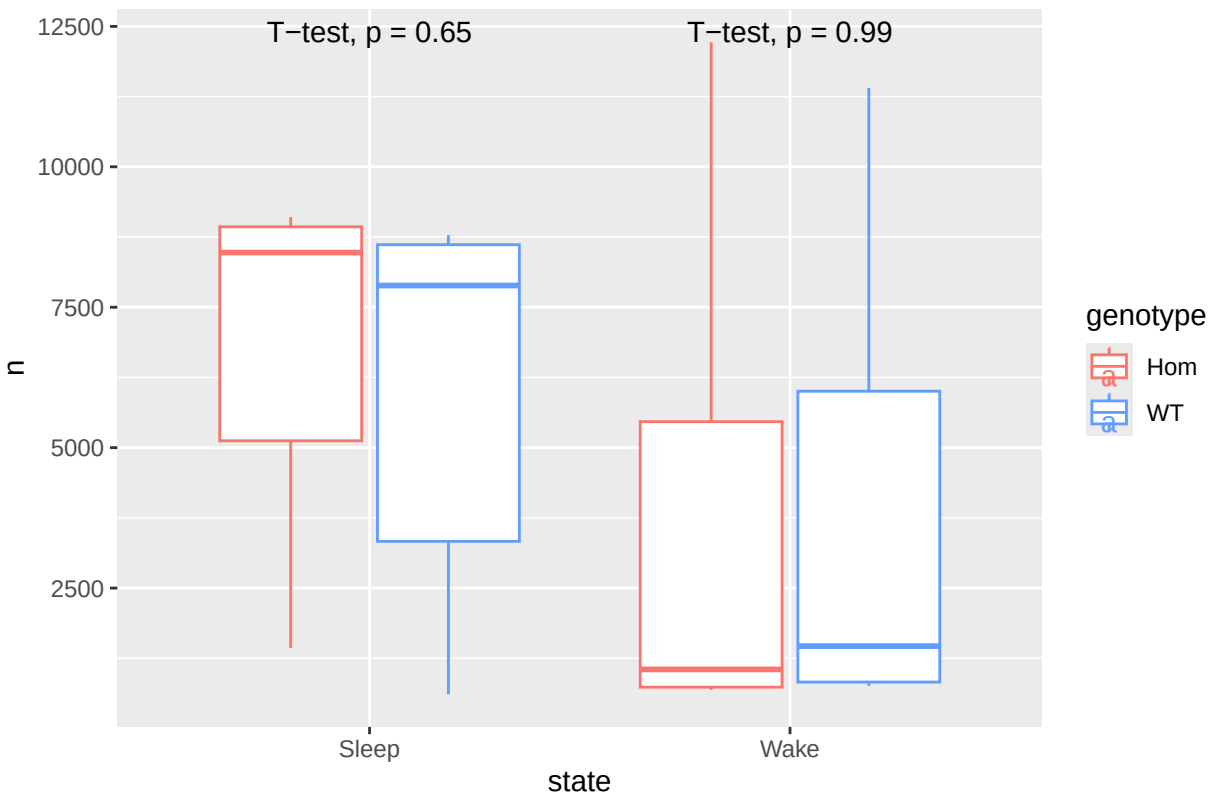
```
fish_counts %>% filter(hour %in% 9:23, genotype %in% c("WT", "Hom")) %>% ggplot(., aes(x = state, y = n
```



We only see this difference during the day and not during the night

```
fish_counts %>% filter(hour %in% 0:8, genotype %in% c("WT", "Hom")) %>% ggplot(., aes(x = state, y = n
```

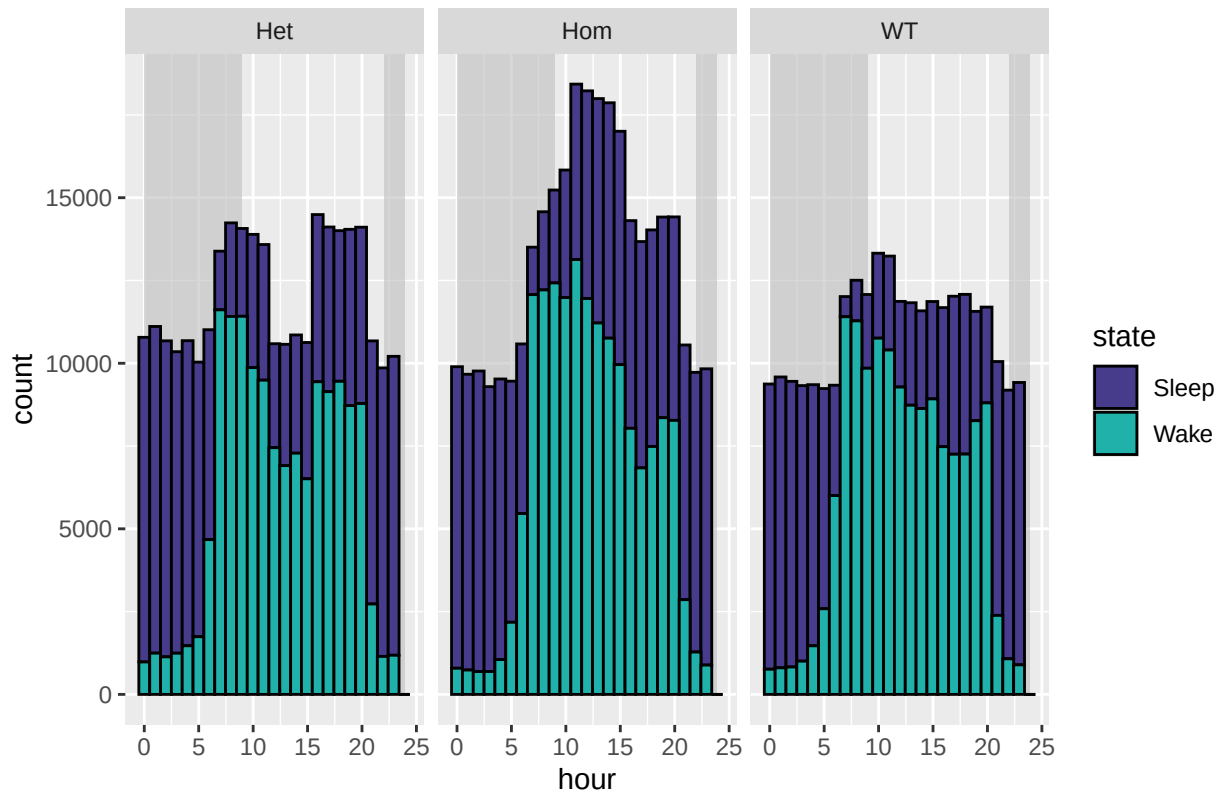
Plot 9: Sleep and wake proportions in wildtype and knockouts during the day



When we look at the total counts (not the proportion) we can see the same result. There are more sleep observations in the daytime for the homozygous mutants compared to wildtype.

```
ggplot(fish_df, aes(x = hour, fill = state)) + annotate("rect", xmin = 0, xmax = 9, ymin = 0, ymax = 10000, fill = "white", stroke = "black")
```


Plot 10: Total wake vs sleep observations in day vs night by genotype



There is no obvious difference in sleep consolidation. Knockouts do not have more fragmented sleep compared to wildtype.

```
fish_df %>% filter(day %in% c(10, 14, 19), hour %in% 0:8) %>% ggplot(., aes(x = time, fill = state)) +
```

Plot 11: Zebrafish sleep is equally consolidated in wildtype as knockouts

